

Tarefa 11: Particionamento de Dados e Balanceamento de Carga

Difusão Viscosa 2D com OpenMP — schedule e collapse

Professor: Samuel Xavier de Souza

Aluno: Vinícius Silva do Carmo

Data: Setembro de 2026

1. Objetivo

Simular a difusão viscosa de um fluido (Navier-Stokes simplificado) em grade 2D usando diferenças finitas, validar a estabilidade e a correção do esquema explícito, e paralelizar o stencil com OpenMP, comparando o impacto das cláusulas schedule (static, dynamic, guided) e collapse no desempenho.

2. Ambiente de Execução

Item	Valor
CPU	AMD Ryzen 5 5600G (4 núcleos)
Grade	512 × 512 células
Passos de tempo (validação)	2000
Passos de tempo (benchmark)	1000
Compilador	GCC -O2 -fopenmp -Wall -Wextra
Sistema Operacional	Linux (WSL2)

3. Modelo Numérico

Desconsiderando pressão e forças externas, a equação de Navier-Stokes se reduz à equação de difusão $\partial u / \partial t = \nu \nabla^2 u$, discretizada por diferenças finitas explícitas (Euler Forward) com um stencil de 5 pontos. O parâmetro de estabilidade é $r = \nu \cdot \Delta t / \Delta x^2$; em 2D o esquema exige $r \leq 0,25$. Com $\nu = 0,1$, $\Delta x = 1,0$ e $\Delta t = 0,1$, obtém-se $r = 0,01$, bem abaixo do limite.

$$u[i][j]^{(n+1)} = u[i][j]^n + r * (u[i+1][j] + u[i-1][j] + u[i][j+1] + u[i][j-1] - 4*u[i][j])$$

As bordas são mantidas fixas em zero (condição de Dirichlet) e nunca atualizadas.

4. Implementação Paralela

O stencil é aplicado a cada célula interior em cada passo de tempo. Como cada célula depende apenas do passo anterior, o loop espacial é paralelizável diretamente — cada linha (ou célula, com collapse) é independente das demais dentro do mesmo passo.

4.1 schedule(runtime)

As 510 linhas do loop externo são distribuídas entre as 4 threads. A política de escalonamento é definida em tempo de execução por `omp_set_schedule()`, o que permite comparar static, dynamic e guided com um único corpo de código, sem duplicação:

```
#pragma omp parallel for schedule(runtime)
for (int i = 1; i < NX-1; i++)
    atualiza_linha(u, u_new, i);
```

4.2 collapse(2)

Funde os dois loops em um único espaço de 260.100 iterações, permitindo divisão mais fina entre as threads ao custo de recalculer os índices 2D a partir do índice linear:

```
#pragma omp parallel for schedule(runtime) collapse(2)
for (int i = 1; i < NX-1; i++)
    for (int j = 1; j < NY-1; j++)
        u_new[i][j] = u[i][j] + R*(u[i+1][j] + u[i-1][j]
        + u[i][j+1] + u[i][j-1]
        - 4.0*u[i][j]);
```

4.3 Double buffering

Ao fim de cada passo os ponteiros dos dois grids são trocados, em vez de copiar a grade de volta. Uma versão anterior fazia um `memcpy` de 2 MB por passo: trabalho estritamente serial e com o mesmo custo de banda de memória que o próprio stencil, o que por Amdahl limitava o speedup a cerca de $1,5\times$ com 4 threads. Eliminar essa cópia foi o que permitiu os resultados da seção 6.

5. Validação

5.1 Teste A — Campo uniforme

Interior inicializado em $u = 1,0$ com bordas fixas em 0. O quarto central da grade, distante das bordas, permanece exatamente em 1,0 ao longo de toda a simulação ($\max|u - 1,0| = 0,00e+00$), confirmando que nenhuma instabilidade artificial é introduzida pelo esquema.

5.2 Teste B — Perturbação gaussiana

Campo nulo com uma gaussiana de amplitude 10 e $\sigma_0 = 5$ células no centro. Uma difusão correta tem assinatura analítica conhecida: o pico cai, a massa Σu se conserva (enquanto a perturbação não alcança as bordas) e a largura cresce segundo $\sigma(t) = \sqrt{(\sigma_0^2 + 2vt)}$. Após 2000 passos ($t = 200$):

```
pico:          10.0000 → 3.8546   (deve cair)
largura:        5.0000 → 8.0623   (analítico: 8.0623)
massa Σu: 1.570796e+03 → 1.570796e+03 (deve se conservar)
mínimo final: 0.00e+00 (≥ 0 → difusão suave, sem oscilação)
```

A largura medida coincide com a solução analítica em quatro casas decimais. Isso valida o esquema numérico em si, e não apenas a ausência de divergência: um erro no stencil que apenas atenuasse o campo passaria por um teste de energia, mas não por este. O mínimo não-negativo confirma que a difusão é suave, sem oscilações espúrias.

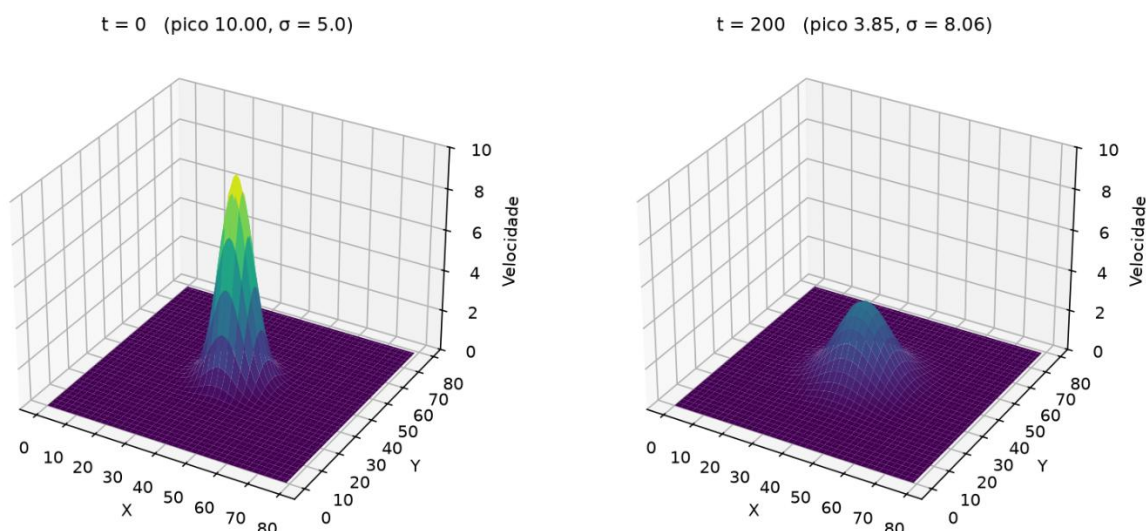


Figura 1 — A perturbação no início e após 2000 passos (recorte de 80×80 células em torno do centro). O pico cai de 10,00 para 3,85 enquanto a gaussiana se alarga, sem oscilações.

6. Resultados

Grade 512×512 , 1000 passos de tempo, 4 threads, mediana de 3 execuções:

Versão	Tempo (s)	Speedup
Sequencial	0,226	1,00
static	0,110	2,05
static, chunk=32	0,080	2,82
dynamic, chunk=16	0,089	2,53
guided	0,076	2,96
collapse(2) + static	0,117	1,93

Entre execuções, os três schedules de granularidade fina (static com chunk, dynamic e guided) oscilam na faixa de $2,5\times$ a $3,1\times$ e trocam de posição entre si — a diferença entre eles está dentro do ruído de medição. O que se mantém consistente é a separação em dois grupos: esses três ficam acima, enquanto static puro e collapse(2) ficam abaixo.

7. Análise

static divide as 510 linhas em 4 blocos contíguos; o acesso row-major favorece o cache e o overhead de coordenação é mínimo. Ainda assim fica em $\sim 2,0\times$, porque o kernel é memory-bound: lê 5 valores e escreve 1 por célula, com pouquíssima aritmética. O gargalo é a banda de memória, não a CPU, e quatro threads disputam o mesmo controlador — daí o teto de aproximadamente $3\times$ observado em todas as variantes.

static com chunk=32 distribui blocos de 32 linhas de forma cíclica entre as threads. O intercalamento faz com que elas percorram regiões mais próximas da memória ao mesmo tempo, melhorando o reuso de cache e o prefetch: $\sim 2,8\times$.

dynamic (chunk=16) e guided atingem $\sim 2,5\times$ e $\sim 3,0\times$. É importante notar que esse ganho não vem do balanceamento de carga: o custo por linha deste stencil é rigorosamente uniforme, de modo que não há desbalanceamento a corrigir e o overhead de coordenação seria, em princípio, puro custo. O que os separa do static puro é o tamanho do chunk — o mesmo efeito de localidade observado no static com chunk explícito. Numa carga genuinamente irregular a vantagem do guided seria estrutural; aqui os três empatam dentro do ruído.

collapse(2) fica em $\sim 1,9\times$, o pior resultado paralelo. O loop interno deixa de ser um laço contíguo simples e passa a exigir a reconstrução dos índices i e j a partir do índice linear colapsado, o que atrapalha a vetorização e o prefetch. Com 510 linhas para 4 threads já há paralelismo de sobra no loop externo: collapse não acrescenta nada e apenas cobra o custo.

8. Conclusão

Cláusula	Speedup	Quando usar
static	$\sim 2,0\times$	Carga uniforme; menor overhead, chunks grandes
static, chunk	$\sim 2,8\times$	Carga uniforme, chunk ajustado ao cache
dynamic	$\sim 2,5\times$	Carga variável (aqui o ganho é de localidade)
guided	$\sim 3,0\times$	Carga variável com cauda longa de iterações
collapse(2)+static	$\sim 1,9\times$	Loop externo com poucas iterações por thread

A principal lição do experimento é que a carga deste stencil já é perfeitamente uniforme: cada linha custa o mesmo, então dynamic e guided não têm desbalanceamento para corrigir, e o que de fato os diferencia do static puro é a granularidade do chunk, não a política de escalonamento. O teto de cerca de $3\times$ com 4 threads é imposto pela banda de memória, não pelo particionamento. O collapse(2) só compensa quando o loop externo oferece menos iterações do que há threads disponíveis — exatamente o oposto do caso aqui. A simulação confirmou ainda que o esquema explícito é estável para $r = 0,01$ e, pela concordância com $\sigma(t)$ analítico, que ele resolve corretamente a física do problema.

9. Código Fonte

```
1  #define _POSIX_C_SOURCE 199309L
2  #include <stdio.h>
3  #include <stdlib.h>
4  #include <math.h>
5  #include <string.h>
6  #include <time.h>
7  #include <omp.h>
8
9  #define NX 512          /* células em x */
10 #define NY 512          /* células em y */
11 #define NU 0.1          /* viscosidade cinemática  $\nu$  */
12 #define DX 1.0          /* espaçamento espacial */
13 #define DT 0.1          /* passo de tempo */
14
15 /*  $r = \nu \cdot \Delta t / \Delta x^2 = 0.01 \leq 0.25 \rightarrow$  esquema explícito estável */
16 #define R (NU * DT / (DX * DX))
17
18 #define NT_VAL 2000      /* passos da validação */
19 #define NT 1000          /* passos do benchmark */
20 #define REPS 3           /* repetições por versão, para tirar a mediana */
21
22 #define AMPLITUDE 10.0   /* pico da perturbação gaussiana */
23 #define SIGMA0 5.0       /* largura inicial da perturbação */
24
25 typedef double (*grid)[NY];
26
27 static grid aloca(void) {
28     grid g = calloc(NX, sizeof *g);
29     if (!g) { perror("calloc"); exit(EXIT_FAILURE); }
30     return g;
31 }
```

```

32
33 /* — Stencil de difusão ————— */
34
35 static inline void atualiza_linha(const double u[NX][NY],
36                                 double u_new[NX][NY], int i) {
37     for (int j = 1; j < NY - 1; j++)
38         u_new[i][j] = u[i][j] + R * (u[i+1][j] + u[i-1][j]
39                                     + u[i][j+1] + u[i][j-1] - 4.0 * u[i][j]);
40 }
41
42 static void passo_seq(const double u[NX][NY], double u_new[NX][NY]) {
43     for (int i = 1; i < NX - 1; i++)
44         atualiza_linha(u, u_new, i);
45 }
46
47 /* schedule(runtime): a política é escolhida em tempo de execução por
48 omp_set_schedule(), o que permite comparar static/dynamic/guided
49 com um único corpo de código. */
50 static void passo_omp(const double u[NX][NY], double u_new[NX][NY]) {
51     #pragma omp parallel for schedule(runtime)
52     for (int i = 1; i < NX - 1; i++)
53         atualiza_linha(u, u_new, i);
54 }
55
56 /* collapse(2) funde os dois laços num espaço único de (NX-2)·(NY-2)
57 iterações, distribuído entre as threads como uma lista só. */
58 static void passo_collapse(const double u[NX][NY], double u_new[NX][NY]) {
59     #pragma omp parallel for schedule(runtime) collapse(2)
60     for (int i = 1; i < NX - 1; i++)
61         for (int j = 1; j < NY - 1; j++)
62             u_new[i][j] = u[i][j] + R * (u[i+1][j] + u[i-1][j]
63                                         + u[i][j+1] + u[i][j-1] - 4.0 * u[i][j]);
64 }
65
66 /* — Evolução temporal ————— */
67
68 typedef void (*passo_fn)(const double[NX][NY], double[NX][NY]);
69
70 /* Recebe os grids por endereço para trocar os ponteiros a cada passo
71 (double buffering). O resultado fica sempre em *u. Retorna o tempo. */
72 static double evoluir(grid *u, grid *u_new, passo_fn passo, int nt) {
73     struct timespec ini, fim;
74     clock_gettime(CLOCK_MONOTONIC, &ini);
75
76     for (int t = 0; t < nt; t++) {
77         passo(*u, *u_new);
78         grid tmp = *u; *u = *u_new; *u_new = tmp;
79     }
80
81     clock_gettime(CLOCK_MONOTONIC, &fim);
82     return (fim.tv_sec - ini.tv_sec) + (fim.tv_nsec - ini.tv_nsec) / 1e9;
83 }
84

```

```

84
85 /* — Condições iniciais ————— */
86
87 static void inicia_uniforme(double u[NX][NY], double valor) {
88     memset(u, 0, NX * NY * sizeof(double));
89     for (int i = 1; i < NX - 1; i++)
90         for (int j = 1; j < NY - 1; j++)
91             u[i][j] = valor;
92 }
93
94 static void inicia_perturbacao(double u[NX][NY], double amp, double sigma) {
95     memset(u, 0, NX * NY * sizeof(double));
96     int ci = NX / 2, cj = NY / 2;
97     for (int i = 1; i < NX - 1; i++)
98         for (int j = 1; j < NY - 1; j++) {
99             double di = i - ci, dj = j - cj;
100             u[i][j] = amp * exp(-(di*di + dj*dj) / (2.0 * sigma * sigma));
101         }
102 }
103
104 /* — Medidas de validação ————— */
105
106 typedef struct {
107     double maximo; /* pico do campo */
108     double minimo; /* < 0 denunciaria oscilação numérica */
109     double massa; /*  $\Sigma u$  – conservada enquanto a perturbação não toca a borda */
110     double sigma; /* largura efetiva:  $\langle r^2 \rangle = 2\sigma^2$  para uma gaussiana 2D */
111 } Perfil;
112
113 static Perfil mede(const double u[NX][NY]) {
114     Perfil p = { -INFINITY, INFINITY, 0.0, 0.0 };
115     double soma_r2 = 0.0;
116     int ci = NX / 2, cj = NY / 2;
117
118     for (int i = 0; i < NX; i++)
119         for (int j = 0; j < NY; j++) {
120             double v = u[i][j];
121             if (v > p.maximo) p.maximo = v;
122             if (v < p.minimo) p.minimo = v;
123             p.massa += v;
124             double di = i - ci, dj = j - cj;
125             soma_r2 += (di*di + dj*dj) * v;
126         }
127
128     p.sigma = sqrt(soma_r2 / (2.0 * p.massa));
129     return p;
130 }
131

```

```

132 /* Maior desvio em relação ao valor esperado no quarto central da grade,
133 região distante o bastante das bordas Dirichlet. */
134 static double max_desvio_centro(const double u[NX][NY], double esperado) {
135     double d = 0.0;
136     for (int i = NX/4; i < 3*NX/4; i++)
137         for (int j = NY/4; j < 3*NY/4; j++) {
138             double diff = fabs(u[i][j] - esperado);
139             if (diff > d) d = diff;
140         }
141     return d;
142 }
143
144 /* — Benchmark ————— */
145
146 static int compara(const void *a, const void *b) {
147     double x = *(const double *)a, y = *(const double *)b;
148     return (x > y) - (x < y);
149 }
150
151 static double mediana(double v[], int n) {
152     qsort(v, n, sizeof *v, compara);
153     return v[n / 2];
154 }
155
156 /* — main ————— */
157
158 int main(void) {
159     grid u = aloca(), u_new = aloca();
160
161     printf("Difusão viscosa (Navier-Stokes só com viscosidade)\n");
162     printf("Grade %dx%d | r = v·Δt/Δx² = %.4f (limite de estabilidade: 0.25)\n\n",
163           NX, NY, R);
164
165     /* — Fase 1: validação ————— */
166     printf("=== Fase 1: validação (%d passos, t = %.1f) ===\n\n",
167           NT_VAL, NT_VAL * DT);
168
169     /* A) Campo uniforme deve permanecer estável. As bordas fixas em zero
170 drenam a periferia, mas o interior não pode se mover. */
171     inicia_uniforme(u, 1.0);
172     evoluir(&u, &u_new, passo_seq, NT_VAL);
173     printf("A) Campo uniforme u = 1.0\n");
174     printf("    max|u - 1.0| no quarto central: %.2e (esperado: ~0)\n\n",
175           max_desvio_centro(u, 1.0));
176
177     /* B) Uma perturbação gaussiana deve se espalhar suavemente: o pico cai,
178 a largura cresce como σ(t) = sqrt(σ₀² + 2vt) e nada oscila. */
179     inicia_perturbacao(u, AMPLITUDE, SIGMA0);
180     Perfil antes = mede(u);
181     evoluir(&u, &u_new, passo_seq, NT_VAL);
182     Perfil depois = mede(u);
183     double sigma_teorico = sqrt(SIGMA0*SIGMA0 + 2.0 * NU * (NT_VAL * DT));
184

```

```

184
185     printf("B) Perturbação gaussiana (amplitude %.1f,  $\sigma_0 = %.1f$ )\n", AMPLITUDE, SIGMA0);
186     printf("    pico:    %8.4f → %8.4f    (deve cair)\n", antes.maximo, depois.maximo);
187     printf("    largura: %8.4f → %8.4f    (analítico: %.4f)\n",
188           antes.sigma, depois.sigma, sigma_teorico);
189     printf("    massa  $\Sigma$ : %.6e → %.6e    (deve se conservar)\n",
190           antes.massa, depois.massa);
191     printf("    mínimo final: %.2e ( $\geq 0 \rightarrow$  difusão suave, sem oscilação)\n\n",
192           depois.minimo);
193
194     /* — Fase 2: schedule e collapse ————— */
195     printf("=== Fase 2: desempenho (%d passos, %d threads, mediana de %d execuções) ===\n\n",
196           NT, omp_get_max_threads(), REPS);
197
198     struct { const char *nome; passo_fn passo; omp_sched_t sched; int chunk; }
199     versoes[] = {
200         { "Sequencial",          passo_seq,      omp_sched_static,  0 },
201         { "static",              passo_omp,      omp_sched_static,  0 },
202         { "static, chunk=32",     passo_omp,      omp_sched_static, 32 },
203         { "dynamic, chunk=16",    passo_omp,      omp_sched_dynamic, 16 },
204         { "guided",              passo_omp,      omp_sched_guided,  0 },
205         { "collapse(2) + static", passo_collapse, omp_sched_static,  0 },
206     };
207     int nv = sizeof versoes / sizeof *versoes;
208     double t_seq = 0.0;
209
210     printf("%-22s %10s %8s\n", "Versão", "Tempo(s)", "Speedup");
211     printf("%-22s %10s %8s\n", "-----", "-----", "-----");
212
213     for (int v = 0; v < nv; v++) {
214         omp_set_schedule(versoes[v].sched, versoes[v].chunk);
215
216         double tempos[REPS];
217         for (int r = 0; r < REPS; r++) {
218             inicia_perturbacao(u, AMPLITUDE, SIGMA0);
219             tempos[r] = evoluir(&u, &u_new, versoes[v].passo, NT);
220         }
221
222         double t = mediana(tempos, REPS);
223         if (v == 0) t_seq = t;
224         printf("%-22s %10.4f %8.2f\n", versoes[v].nome, t, t_seq / t);
225     }
226
227     free(u);
228     free(u_new);
229     return 0;
230 }
231

```